

# Compiler Pipeline Visualizer - Software Architecture Project

## Compiler Pipeline Visualizer - Software Architecture Project

### Complete Work Split (4-Person Team, 2 Phases)

Team:

- **Backend:** Ebin, Kevin
  - **Frontend:** Aadi, Joel
  - **Tech Stack:** Next.js, Neon Database, TypeScript, React
- 

### Project Overview

Build an interactive compiler pipeline visualizer that allows students to:

- Input source code and visualize each compilation stage
  - Inspect symbol tables, tokens, and error logs
  - Understand lexical analysis → syntax analysis → semantic analysis → intermediate code → optimization → target code generation
- 

## ◆ PHASE 1: Core Infrastructure & Foundation (Weeks 1-3)

### Backend Tasks (Ebin & Kevin)

#### Ebin - Core Compiler Engine & Database Setup

##### 1. Project Setup & Database Schema (Days 1-2)

- ☐ Initialize Next.js project with TypeScript
- ☐ Setup Neon PostgreSQL database
- ☐ Create database schema:

```
- compilation_sessions (id, user_id, source_code, created_at)-
lexical_tokens (id, session_id, type, value, line, column)- syntax_tree
(id, session_id, tree_json)- symbol_table (id, session_id, name, type,
scope, metadata)- semantic_info (id, session_id, error_type, message, line,
column)- intermediate_code (id, session_id, ir_code, line_number)-
compilation_errors (id, session_id, phase, error_msg, severity)
```

- ☐ Setup Neon connection with connection pooling
- ☐ Create migration scripts

## 2. Lexical Analyzer Implementation (Days 3-4)

- ☐ Implement tokenization engine
  - ☐ Token types definition (KEYWORD, IDENTIFIER, OPERATOR, LITERAL, etc.)
  - ☐ Regex-based pattern matching
  - ☐ Handle whitespace and comments
  - ☐ Line/column tracking for error reporting
- ☐ Create `LexicalAnalyzer` class
- ☐ API endpoint: `POST /api/compile/lex`
  - Input: source code
  - Output: token array with positions
- ☐ Store tokens in database
- ☐ Error handling for invalid characters

## 3. Symbol Table & Data Structures (Days 5-6)

- ☐ Implement `SymbolTable` class with scoping
  - ☐ Enter/exit scope functionality
  - ☐ Symbol insertion and lookup
  - ☐ Scope chain management (global → local)
- ☐ Implement `Token` class
- ☐ Implement `ASTNode` base class for syntax tree nodes
- ☐ Create utility functions for type checking
- ☐ Database operations for symbol table persistence

## 4. API Route Setup & Testing (Day 7)

- ☐ Create API folder structure: `/api/compile/*`
- ☐ Setup error handling middleware
- ☐ Create logging system
- ☐ Unit tests for lexer and symbol table

- ☐ Basic integration tests
- 

## Kevin - Syntax & Semantic Analysis Foundation

### 1. Syntax Analyzer Implementation (Days 1-4)

- ☐ Implement recursive descent parser
  - ☐ Grammar definition for simple C-like language
  - ☐ Expression parsing (binary ops, unary ops, function calls)
  - ☐ Statement parsing (if, while, for, assignments)
  - ☐ Function and variable declarations
- ☐ Create `SyntaxAnalyzer` class
- ☐ Build abstract syntax tree (AST)
- ☐ Implement error recovery strategies
- ☐ API endpoint: `POST /api/compile/syntax`
  - Input: tokens from lexer
  - Output: AST structure
- ☐ Store AST as JSON in database

### 2. Semantic Analysis Setup (Days 5-6)

- ☐ Design semantic analyzer architecture
- ☐ Implement type checking system
  - ☐ Basic types: int, float, string, bool
  - ☐ Function signature validation
  - ☐ Variable redeclaration detection
- ☐ Create `SemanticAnalyzer` class
- ☐ Implement scope resolution
- ☐ API endpoint: `POST /api/compile/semantic`
  - Input: AST + Symbol table
  - Output: Annotated AST with type info + errors

### 3. Error Management System (Day 7)

- ☐ Create comprehensive error tracking
  - ☐ Error types: Syntax, Type, Semantic, Undefined symbol
  - ☐ Error messages with context
  - ☐ Error severity levels (ERROR, WARNING)

- ☐ Database schema for error storage
  - ☐ Error retrieval API endpoint: GET /api/compile/:sessionId/errors
- 

## Frontend Tasks (Aadi & Joel)

### Aadi - UI Layout, Code Editor & Input Management

#### 1. Project Setup & Component Structure (Days 1-2)

- ☐ Initialize Next.js frontend structure
- ☐ Setup Tailwind CSS for styling
- ☐ Create component folder structure:

```
components/ - CodeEditor/ - Pipeline/ - SymbolTable/ - TokenList/ -  
ErrorPanel/ - common/
```

- ☐ Setup global state management (Redux/Zustand)
- ☐ Create environment config for API endpoints

#### 2. Code Editor Component (Days 3-5)

- ☐ Integrate Monaco Editor or CodeMirror
  - ☐ Syntax highlighting for C-like language
  - ☐ Line numbers and indentation
  - ☐ Theme support (light/dark)
  - ☐ Font size adjustment
- ☐ Create CodeEditor component
- ☐ Add keyboard shortcuts (Ctrl+Enter to compile)
- ☐ Store code in global state
- ☐ Local storage persistence

#### 3. Control Panel & Input Management (Day 6)

- ☐ Create compilation control buttons
  - ☐ "Compile" button
  - ☐ "Clear All" button
  - ☐ "Load Example" dropdown
- ☐ Sample code templates:
  - ☐ Simple variable declaration
  - ☐ Function with parameters

- ☐ Loop examples
- ☐ Conditional statements
- ☐ Display compilation status (loading, success, error)

## 4. Responsive Layout (Day 7)

- ☐ Design 2-column layout (Editor on left, Output on right)
  - ☐ Responsive breakpoints for mobile
  - ☐ Create Layout component wrapper
  - ☐ Navigation/header component
- 

# Joel - Data Visualization Components & State Flow

## 1. Pipeline Visualization Framework (Days 1-3)

- ☐ Design visualization data structure
- ☐ Create `PipelineVisualizer` component (main container)
- ☐ Implement phase cards layout
  - ☐ Card for each compilation phase
  - ☐ Phase name, status indicator (pending/processing/complete)
  - ☐ Show input/output for each phase
- ☐ Add transition animations between phases
- ☐ Create progress indicator

## 2. Token & Syntax Tree Display (Days 4-5)

- ☐ Create `TokenList` component
  - ☐ Table displaying: Token Type, Value, Line, Column
  - ☐ Syntax highlighting for token types
  - ☐ Search/filter tokens by type
- ☐ Create `SyntaxTreeViewer` component
  - ☐ Tree structure visualization
  - ☐ Collapsible nodes
  - ☐ Show node properties (type, value)
  - ☐ Highlight syntax nodes

## 3. Symbol Table & Info Panels (Days 6-7)

- ☐ Create `SymbolTableViewer` component

- ☐ Display: Name, Type, Scope, Value
  - ☐ Filter by scope
  - ☐ Sort functionality
  - ☐ Create `InfoPanel` component
    - ☐ Display metadata for selected items
    - ☐ Show scope information
    - ☐ Context-aware information
- 

## ◆ Phase 1 Deliverables

### Backend

- ☒ Database schema and migrations
- ☒ Lexical analyzer with API endpoint
- ☒ Syntax analyzer with API endpoint
- ☒ Semantic analyzer skeleton
- ☒ Symbol table management system
- ☒ Error tracking system
- ☒ Core compilation pipeline logic

### Frontend

- ☒ Code editor with syntax highlighting
- ☒ Phase-based pipeline visualization
- ☒ Token list display
- ☒ Basic symbol table viewer
- ☒ Responsive layout structure
- ☒ Sample code templates

### Testing & Documentation

- ☒ Backend unit tests (20+ test cases)
  - ☒ API documentation
  - ☒ Component documentation
- 
-

## ◆ PHASE 2: Advanced Features & Polish (Weeks 4-6)

### Backend Tasks (Ebin & Kevin)

#### Ebin - Intermediate Code Generation & Optimization

##### 1. Intermediate Code Generation (Days 1-3)

- ☐ Implement `IntermediateCodeGen` class
  - ☐ Three-address code (TAC) generation
  - ☐ Temporary variable management
  - ☐ Label and jump instruction generation
- ☐ Support compilation constructs:
  - ☐ Assignments and expressions
  - ☐ If-else statements
  - ☐ Loops (while, for)
  - ☐ Function calls
- ☐ Create intermediate code API endpoint: `POST /api/compile/intermediate`
  - Input: Semantic AST
  - Output: TAC/intermediate code with line mappings
- ☐ Store intermediate code in database
- ☐ Implement code pretty-printing for visualization

##### 2. Code Optimization Phase (Days 4-5)

- ☐ Implement `Optimizer` class
  - ☐ Dead code elimination
  - ☐ Constant folding
  - ☐ Copy propagation
  - ☐ Common subexpression elimination (CSE)
- ☐ Create optimization report showing:
  - ☐ Before/after code comparison
  - ☐ Optimization applied
  - ☐ Size reduction
- ☐ API endpoint: `POST /api/compile/optimize`

##### 3. Target Code Generation (Days 6-7)

- ☐ Implement `TargetCodeGen` class
  - ☐ Simple assembly-like target language

- ☐ Register allocation (simple approach)
  - ☐ Instruction selection
  - ☐ Label resolution
  - ☐ API endpoint: `POST /api/compile/target`
    - Input: Optimized intermediate code
    - Output: Target code representation
  - ☐ Create execution simulation (optional)
- 

## Kevin - Register Allocation, Parallelization & Advanced Analysis

### 1. Register Allocator (Days 1-3)

- ☐ Implement `RegisterAllocator` class
  - ☐ Build live variable analysis
  - ☐ Create interference graph
  - ☐ Simple graph coloring for allocation
  - ☐ Spill handling
- ☐ Display allocation results:
  - ☐ Variable to register mapping
  - ☐ Spill decisions
  - ☐ Register pressure visualization

### 2. Advanced Error Detection & Reporting (Days 4-5)

- ☐ Implement comprehensive error categories:
  - ☐ Type mismatch errors
  - ☐ Undefined variable/function references
  - ☐ Array bounds checking (static analysis)
  - ☐ Division by zero (constant analysis)
  - ☐ Dead code warnings
- ☐ Enhanced error messages with suggestions
- ☐ Create `ErrorAnalyzer` class
- ☐ Error location mapping back to source code

### 3. Compilation History & Session Management (Days 6-7)

- ☐ Implement session persistence
  - ☐ Save/load compilation sessions

- ☐ Track compilation history with timestamps
  - ☐ Create history database schema extensions
  - ☐ Create history API endpoints:
    - ☐ GET /api/sessions - list all sessions
    - ☐ GET /api/sessions/:id - get session details
    - ☐ DELETE /api/sessions/:id - delete session
  - ☐ Export compilation results (JSON, PDF)
- 

## Frontend Tasks (Aadi & Joel)

### Aadi - Error Panel, Control Panel, & State Management

#### 1. Error Panel & Logging (Days 1-3)

- ☐ Create `ErrorPanel` component
  - ☐ Display errors with severity color-coding
  - ☐ Show error type, message, line number
  - ☐ Click error to jump to code line
  - ☐ Filter errors by type/severity
  - ☐ Show error suggestions/fixes
- ☐ Implement error source mapping (highlight in editor)
- ☐ Create error statistics/summary

#### 2. Advanced Control Panel (Days 4-5)

- ☐ Create compilation flow controls:
  - ☐ Step-by-step compilation button
  - ☐ Run to specific phase button
  - ☐ Pause/Resume buttons
  - ☐ Speed control for animations
- ☐ Create export functionality:
  - ☐ Export as JSON
  - ☐ Export as PDF report
  - ☐ Share compilation session
- ☐ Implement settings panel:
  - ☐ Theme (light/dark)
  - ☐ Font size

- ☐ Animation speed
- ☐ Auto-compile toggle

### 3. Session Management UI (Days 6-7)

- ☐ Create history panel
    - ☐ List previous compilations
    - ☐ Load previous session
    - ☐ Delete session with confirmation
    - ☐ Show metadata (date, code length, status)
  - ☐ Create session browser component
  - ☐ Implement session comparison view
- 

## Joel - Advanced Visualization & Interactive Features

### 1. Enhanced Pipeline Visualization (Days 1-3)

- ☐ Create animated phase flow
  - ☐ Show data flow between phases
  - ☐ Animated arrows showing data movement
  - ☐ Phase execution time display
  - ☐ Highlight errors in specific phases
- ☐ Create detailed phase views:
  - ☐ Expandable phase cards
  - ☐ Show input/output data with formatting
  - ☐ Transformation visualization
- ☐ Create zoom and pan functionality

### 2. Intermediate Code & Target Code Viewer (Days 4-5)

- ☐ Create `IntermediateCodeViewer` component
  - ☐ Display TAC/intermediate code
  - ☐ Syntax highlighting
  - ☐ Line numbering
  - ☐ Show optimization transformations
- ☐ Create `TargetCodeViewer` component
  - ☐ Display assembly-like target code
  - ☐ Memory layout visualization

- ☐ Register allocation table
- ☐ Side-by-side comparison with intermediate code

### 3. Interactive Analysis & Debugging Features (Days 6-7)

- ☐ Create `DebuggerPanel` component
    - ☐ Variable value tracking
    - ☐ Step-through execution (if applicable)
    - ☐ Variable state at each phase
    - ☐ Memory visualization
  - ☐ Create code coverage/complexity display
  - ☐ Implement interactive tutorials/tooltips
    - ☐ Phase explanations
    - ☐ Compiler concept guides
- 

## ◆ Phase 2 Deliverables

### Backend

- ☒ Intermediate code generation
- ☒ Code optimization system
- ☒ Target code generation
- ☒ Register allocation
- ☒ Advanced error detection
- ☒ Session management and history
- ☒ Export functionality
- ☒ Complete API documentation

### Frontend

- ☒ Error panel with line mapping
- ☒ Advanced control panel with step-by-step execution
- ☒ Session management UI
- ☒ Enhanced pipeline visualization with animations
- ☒ Intermediate code viewer
- ☒ Target code viewer
- ☒ Interactive debugging features
- ☒ Settings/preferences panel

## Testing & Polish

- ☒ End-to-end testing (20+ scenarios)
  - ☒ Performance optimization
  - ☒ Cross-browser testing
  - ☒ Responsive design refinement
  - ☒ Accessibility improvements (WCAG)
  - ☒ User documentation and tutorials
  - ☒ Demo video/presentation
- 
- 



## Task Summary by Person

### Ebin (Backend Lead)

#### Phase 1:

- Project setup & database schema
- Lexical analyzer implementation
- Symbol table & data structures
- API setup & testing

#### Phase 2:

- Intermediate code generation
- Code optimization system
- Target code generation

**Total Hours:** ~60-70 hours **Key Deliverables:** Core compiler pipeline, data persistence layer

---

### Kevin (Backend)

#### Phase 1:

- Syntax analyzer implementation
- Semantic analysis foundation
- Error management system

## Phase 2:

- Register allocator
- Advanced error detection
- Session management & history

**Total Hours:** ~60-70 hours **Key Deliverables:** Language parsing, error handling, optimization

---

## Aadi (Frontend)

### Phase 1:

- Project setup & structure
- Code editor component
- Control panel & inputs
- Responsive layout

### Phase 2:

- Error panel implementation
- Advanced control features
- Session management UI

**Total Hours:** ~55-65 hours **Key Deliverables:** User input interface, error visualization

---

## Joel (Frontend)

### Phase 1:

- Pipeline visualization framework
- Token & syntax tree display
- Symbol table viewer

### Phase 2:

- Enhanced visualizations with animations
- Intermediate & target code viewers
- Interactive debugging features

**Total Hours:** ~60-70 hours **Key Deliverables:** Data visualization, interactive components

---

## Integration Points & Collaboration

### Ebin ↔ Kevin (Backend)

- Daily standup on compiler logic consistency
- Shared schema for AST representation
- Error message standardization
- Testing database operations together

### Aadi ↔ Joel (Frontend)

- State management coordination (Redux/Zustand)
- Component prop interface agreements
- Styling consistency (Tailwind config)
- Responsive breakpoint alignment

### Backend ↔ Frontend (Cross-team)

- **API Contract Definition (End of Phase 1, Start of Phase 2)**
  - Endpoint specifications
  - Request/response schemas
  - Error response formats
  - Authentication/authorization
- **Integration Testing (Weekly)**
  - End-to-end compilation flow
  - Data format validation
  - Error handling
  - Performance testing

### Weekly Sync Points

- **Monday:** Phase planning & blockers
- **Wednesday:** Integration testing & mid-phase check
- **Friday:** Demo & retrospective

---

## Success Criteria & Quality Metrics

### Backend

- ☐ All API endpoints return < 500ms response time
- ☐ Lexer handles 10,000+ token compilations
- ☐ 90%+ test coverage for core modules
- ☐ Proper error recovery (no crash on invalid input)
- ☐ Database queries optimized (< 100ms)

## Frontend

- ☐ < 3 second initial load time
- ☐ Smooth 60fps animations
- ☐ Responsive on mobile (375px width)
- ☐ Accessibility score > 90 (Lighthouse)
- ☐ Code organized with no circular dependencies

## Overall

- ☐ Full compilation pipeline working end-to-end
- ☐ Handles sample programs correctly
- ☐ Professional UI/UX
- ☐ Comprehensive documentation
- ☐ Presentation-ready demo



## Tech Stack Reference

### Backend

- Framework: Next.js 14+ (API routes)
- Language: TypeScript
- Database: Neon PostgreSQL
- ORM: Prisma
- Testing: Jest + Supertest
- Logging: Winston/Pino

### Frontend

- Framework: Next.js 14+ (React components)
- Language: TypeScript
- Styling: Tailwind CSS
- Code Editor: Monaco Editor

- State Management: Zustand/Redux Toolkit
- Data Visualization: Recharts/D3.js
- Testing: Vitest + React Testing Library

## DevOps

- Version Control: Git (GitHub)
- CI/CD: GitHub Actions
- Deployment: Vercel (Next.js native)
- Database Host: Neon



## Timeline Overview

### Week 1-3: Phase 1 (Core Infrastructure)

- Mon: Setup & planning
- Tue-Thu: Active development
- Fri: Integration & testing

### Week 4-6: Phase 2 (Advanced Features)

- Mon: Phase 2 planning
- Tue-Thu: Feature development
- Fri: Polish & testing

### Week 7: Final Testing & Presentation Prep

- Demo preparation
- Documentation finalization
- Performance optimization



## Documentation Requirements

Each phase should include:

- ☐ README with setup instructions
- ☐ API documentation (Swagger/OpenAPI)
- ☐ Component documentation with Storybook
- ☐ Architecture decision records (ADRs)
- ☐ Database schema documentation
- ☐ Deployment guide

---

## **Pre-Project Checklist**

- ☐ GitHub repository created and initialized
- ☐ Neon database provisioned
- ☐ Team coding standards document created
- ☐ Git workflow defined (branching strategy)
- ☐ Development environment setup guide created
- ☐ Communication channels established (Discord/Slack)
- ☐ Weekly meeting schedule confirmed

---

**Good luck with your Compiler Pipeline Visualizer project! 🚀**